UNIVERSIDAD DE GRANADA

UNIVERSIDAD DE GRANADA

Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación (ETSIIT)

Trazas Wireshark

Autor:
Adrián Macías Caballero
Paula Genaro Soria

Asignatura:
Desarrollo de Aplicaciones en Red

18 de marzo de 2026

Índice

1. Introducción **2**

2. Hosts identificados y función de cada uno **2**

3. Puerto del servidor y puertos de cliente **2**

4. Comunicación concurrente de múltiples clientes **3**

5. Establecimiento de las conexiones TCP **4**

6. Comunicación entre cliente y servidor local **4**

7. Resolución DNS observada **5**
7.1. Interpretación de las consultas DNS 5

8. Conexiones HTTP hacia el servicio externo **5**
8.1. Respuesta recibida 6

9. Secuencia funcional observada **6**

10. Análisis del cierre de conexiones **7**

11. Aspectos destacables de la captura **7**

12. Interpretación general del comportamiento del sistema **7**

13. Conclusión **8**

1. Introducción

En este apartado se presenta el análisis de las trazas de red capturadas en la máquina servidor durante la ejecución del sistema. El objetivo de este estudio es identificar cómo se establece la comunicación entre cliente y servidor, qué puertos intervienen, qué papel desempeña cada host observado en la captura y cuál es el flujo general seguido por las peticiones.

La captura ha sido realizada en la máquina del servidor, mientras varios clientes se conectaban y realizaban solicitudes. A partir de las trazas obtenidas en Wireshark, puede observarse tanto la comunicación interna dentro de la red local como las conexiones externas que el cliente realiza hacia un servicio de terceros, en este caso la API de OpenWeather.

2. Hosts identificados y función de cada uno

A partir de la captura se distinguen claramente varios equipos y direcciones IP con roles diferentes:

Cuadro 1: Identificación de hosts observados en la traza

Dirección IP	**Rol**	**Descripción**
172.20.10.6	Servidor local	Máquina donde se realiza la captura. Escucha en el puerto 5000 y atiende las conexiones entrantes del cliente.
172.20.10.7	Cliente	Host que establece conexiones con el servidor local y además realiza peticiones HTTP a un servicio externo.
172.20.10.1	DNS / gateway	Equipo que responde a consultas DNS dentro de la red local.
127.0.0.1 / 127.0.0.53	Resolver local	Resolución DNS local del sistema antes de reenviar o resolver consultas.
141.95.47.140	Servidor externo	IP final resuelta para la API de OpenWeather, accesible por HTTP en el puerto 80.

De esta forma, la arquitectura observada en la traza no es únicamente una comunicación cliente-servidor simple, sino un flujo en el que el cliente consulta primero al servidor local y, además, accede a un servicio web externo para obtener información adicional.

3. Puerto del servidor y puertos de cliente

El puerto de servicio del servidor local es el **5000**. Esto se observa repetidamente en las trazas, donde la IP 172.20.10.6 aparece recibiendo conexiones sobre dicho

puerto.

Por otro lado, el cliente 172.20.10.7 utiliza **puertos efímeros** o temporales para iniciar las conexiones. Entre los que se observan en la captura se encuentran:

- 58524
- 55654
- 58474

Además, cuando el mismo cliente realiza conexiones HTTP hacia el servidor externo 141.95.47.140, vuelve a utilizar nuevos puertos efímeros distintos, por ejemplo:

- 52306
- 39182
- 36778
- 38600
- 38616
- 38618
- 42896
- 42910
- 42924
- 42936
- 39684

Esto es completamente normal en TCP: el servidor escucha siempre en un puerto fijo, mientras que el cliente va abriendo conexiones desde puertos altos elegidos dinámicamente por el sistema operativo.

4. Comunicación concurrente de múltiples clientes

Durante el análisis de la traza se observa que existen varias conexiones activas de manera simultánea entre el cliente y el servidor local. En concreto, el host 172.20.10.7 establece múltiples sesiones TCP concurrentes hacia el servidor 172.20.10.6 en el puerto 5000, utilizando distintos puertos efímeros para cada una de ellas. Entre los más representativos se encuentran los puertos 58524, 55654 y 58474, que pueden interpretarse como tres clientes lógicos o tres peticiones independientes ejecutándose en paralelo.

Cada una de estas conexiones sigue su propio ciclo de vida completo, incluyendo establecimiento mediante three-way handshake, intercambio de datos mediante

segmentos PSH, ACK y cierre ordenado mediante FIN, ACK. La coexistencia de estas conexiones simultáneas indica que el servidor es capaz de gestionar múltiples solicitudes concurrentes sin interferencias, lo cual es característico de aplicaciones cliente-servidor modernas. Además, este comportamiento sugiere que el cliente no reutiliza una única conexión persistente, sino que abre nuevas sesiones TCP para cada interacción, lo que puede deberse a la implementación de la aplicación o al uso de múltiples instancias o hilos de ejecución.

5. Establecimiento de las conexiones TCP

En varias ocasiones se observa claramente el establecimiento correcto de una conexión TCP mediante el conocido *three-way handshake*. El patrón es el siguiente:

1. El cliente envía un paquete `SYN`.
2. El servidor responde con `SYN, ACK`.
3. El cliente confirma con `ACK`.

Un ejemplo claro de ello es la conexión interna:

`172.20.10.7:58524 →172.20.10.6:5000`

donde se aprecia el siguiente intercambio:

- `58524 ->5000 [SYN]`
- `5000 ->58524 [SYN, ACK]`
- `58524 ->5000 [ACK]`

Este comportamiento indica que la conexión se establece correctamente y que no hay rechazo del servicio ni problemas de conectividad entre ambos extremos.

6. Comunicación entre cliente y servidor local

Una vez completado el *handshake*, se observan paquetes marcados como `PSH, ACK`, lo que indica transmisión de datos útiles dentro de una conexión ya establecida.

En la captura, el flujo general con el servidor local es el siguiente:

1. El cliente 172.20.10.7 abre una conexión TCP hacia 172.20.10.6:5000.
2. Se intercambian datos mediante segmentos `PSH, ACK`.
3. El servidor responde y mantiene la sesión activa mientras se completan las peticiones.
4. Finalmente la conexión se cierra mediante segmentos `FIN, ACK` y sus correspondientes `ACK`.

La presencia de múltiples conexiones casi consecutivas desde distintos puertos efímeros del cliente hacia el mismo puerto 5000 del servidor sugiere que la aplicación no reutiliza una única conexión persistente, sino que abre varias sesiones independientes. Esto puede deberse a distintos accesos concurrentes, refrescos de interfaz, varias instancias del cliente o una implementación basada en peticiones separadas.

7. Resolución DNS observada

Un aspecto importante de la captura es la resolución del dominio:

`api.openweathermap.org`

Se observa que antes de establecer la conexión HTTP con el servidor externo, el cliente realiza consultas DNS de tipo `A` y `AAAA`. La respuesta muestra que:

`api.openweathermap.org` → `eu-api.openweathermap.org` → `141.95.47.140`

Es decir, el nombre original no resuelve directamente a una dirección IP, sino que primero pasa por un alias (*CNAME*) y finalmente se obtiene la dirección IPv4 del servidor externo.

También aparecen respuestas `SOA` en las consultas AAAA, lo que indica que no se está devolviendo una dirección IPv6 útil para esa consulta concreta, o que la resolución para IPv6 no está disponible en ese contexto.

7.1. Interpretación de las consultas DNS

El patrón observado es coherente con el funcionamiento habitual de una aplicación cliente moderna:

- Se intenta resolver tanto IPv4 (`A`) como IPv6 (`AAAA`).
- La aplicación termina usando IPv4, ya que posteriormente la conexión se realiza contra `141.95.47.140`.
- La consulta DNS se repite varias veces, lo que puede indicar ausencia de caché efectiva, expiración rápida o que cada petición se lanza de forma independiente.

8. Conexiones HTTP hacia el servicio externo

Tras la resolución DNS, el cliente 172.20.10.7 establece conexiones TCP con el host 141.95.47.140 sobre el puerto 80, lo que indica tráfico HTTP sin cifrar.

Un ejemplo observado en la captura es:

`172.20.10.7:52306 →141.95.47.140:80`

El flujo vuelve a seguir el patrón habitual:

- `52306 ->80 [SYN]`
- `80 ->52306 [SYN, ACK]`
- `52306 ->80 [ACK]`

A continuación se envía una petición HTTP GET con una URL del tipo:

```
/data/2.5/weather?q=Granada&appid=...&units=metric
```

o bien peticiones similares para otras ciudades como Madrid o Valdepeñas, tal y como se aprecia en las trazas.

8.1. Respuesta recibida

El servidor externo responde con:

```
HTTP/1.1 200 OK
```

y el contenido se identifica como:

```
application/json
```

Esto confirma que la solicitud ha sido procesada correctamente y que el cliente recibe datos en formato JSON, previsiblemente con información meteorológica.

9. Secuencia funcional observada

A nivel lógico, el comportamiento completo del sistema, según la traza, puede resumirse en la siguiente secuencia:

1. El cliente abre una conexión con el servidor local en el puerto 5000.
2. Se intercambian datos entre ambos.
3. El cliente resuelve el nombre `api.openweathermap.org`.
4. El cliente abre una conexión HTTP con la IP externa resuelta.
5. Envía una petición GET para consultar datos meteorológicos.
6. Recibe una respuesta `200 OK` con contenido JSON.
7. La conexión externa se cierra.
8. Continúa o finaliza la comunicación con el servidor local.

Este patrón se repite varias veces durante la captura, lo que indica un comportamiento reiterativo y estable del sistema.

10. Análisis del cierre de conexiones

En la captura se observan cierres mediante paquetes:

- `FIN, ACK`
- `ACK`

Este detalle es importante porque indica que el cierre de la conexión se produce de forma ordenada y correcta. No se observan reinicios bruscos mediante paquetes `RST`, lo que sugiere que no hay errores graves de transporte ni finalizaciones abruptas de sesión.

Por tanto, desde el punto de vista de TCP, la comunicación puede considerarse estable y correctamente gestionada.

11. Aspectos destacables de la captura

A partir de la observación completa de las trazas se pueden extraer varias conclusiones parciales relevantes:

- El servidor local escucha en el puerto 5000 y responde correctamente a las conexiones entrantes.
- El cliente utiliza puertos efímeros distintos para cada conexión, tanto hacia el servidor local como hacia el servicio externo.
- La resolución DNS se produce correctamente y desemboca en la IP pública 141.95.47.140.
- Las peticiones HTTP obtienen respuesta satisfactoria (`200 OK`).
- El contenido devuelto por el servicio externo es JSON, lo que encaja con una API REST.
- No se aprecian errores de conexión, rechazos, retransmisiones problemáticas ni cierres anómalos.
- El uso del puerto 80 implica que la comunicación externa se realiza en HTTP y no en HTTPS, por lo que los datos viajan sin cifrar.

12. Interpretación general del comportamiento del sistema

La traza sugiere un sistema en el que el cliente no se limita a enviar y recibir mensajes con el servidor local, sino que además realiza consultas externas a una API web. Esto podría significar varias cosas, según cómo esté implementada la práctica:

- que el cliente obtiene información directamente de Internet y después la utiliza en la interacción con el servidor;
- que el servidor local coordina parte del proceso, pero no centraliza todas las peticiones;
- o que se trata de una aplicación distribuida en la que conviven comunicación local y consumo de servicios externos.

En cualquier caso, desde el punto de vista de red, el comportamiento observado es coherente y funcional.

13. Conclusión

El análisis de la captura permite afirmar que la comunicación entre los nodos implicados funciona correctamente. El servidor local, situado en la dirección 172.20.10.6, escucha en el puerto 5000 y atiende las conexiones del cliente 172.20.10.7 sin errores aparentes. El cliente, por su parte, utiliza distintos puertos efímeros para abrir varias conexiones TCP independientes.

Además de la comunicación interna, la traza muestra que el cliente resuelve el dominio `api.openweathermap.org` mediante DNS y establece conexiones HTTP con el servidor externo `141.95.47.140`, al que realiza peticiones GET para obtener datos meteorológicos. Todas estas solicitudes reciben respuestas válidas con código `200 OK`, lo que confirma el funcionamiento correcto del intercambio.

En conjunto, las trazas reflejan un flujo de comunicación bien estructurado: establecimiento de conexión TCP, intercambio de datos, resolución DNS, acceso a un servicio externo, recepción de respuesta y cierre ordenado de las sesiones. No se observan fallos graves ni comportamientos anómalos, por lo que puede concluirse que, dentro del intervalo capturado, el sistema se comporta de forma estable y conforme a lo esperado.

Resumen breve para incluir en el README o memoria

Durante la captura realizada en el servidor se ha observado que la máquina 172.20.10.6 actúa como servidor local escuchando en el puerto 5000, mientras que 172.20.10.7 se comporta como cliente y establece múltiples conexiones TCP usando puertos efímeros. Además, el cliente realiza consultas DNS para resolver el dominio `api.openweathermap.org` y posteriormente abre conexiones HTTP con la IP pública 141.95.47.140 para solicitar datos meteorológicos. Las respuestas recibidas son correctas (`HTTP/1.1 200 OK`) y las conexiones se cierran de forma ordenada, por lo que no se aprecian errores de comunicación en la traza analizada.